
Think and Slide

By

Syed Sami Sayem	ID: 202451003247
MD Tanvir Islam	ID: 202451003244
Raisa Sadik Smrity	ID: 202451003248
Sourav Chandra Shil	ID: 202451003254
Tasmia Anwar Nisa	ID: 202451003261

Bachelor of Science in Computer Science and Engineering



Department of Computer Science and Engineering
Bangladesh University of Business and Technology

August 2026

Declaration

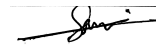
We do hereby declare that the software development project presented in this report entitled “Think and Slide” is the result of our own work. The application was designed and implemented as a Flutter-based mobile game with Firebase-backed user and progress services. We further declare that this report has been compiled and written by us for the academic requirements of the Software Development Project course. Materials, libraries, frameworks, and external resources used in the project have been acknowledged appropriately.

Signature of Authors



1st group member name

ID: 20245103261



2nd group member name

ID: 20245103247



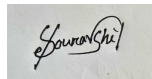
3rd group member name

ID: 20245103244



4th group member name

ID: 20245103248



5th group member name

ID: 20245103254

Approval

It is hereby certified that the software development project presented in this report entitled “Think and Slide” has been developed by the listed group members as part of the requirements for the Bachelor of Science in Computer Science and Engineering. The report describes the analysis, design, implementation, testing, and evaluation of the completed application and is considered suitable for academic evaluation.

Md. Ashiqur Rahman

Assistant Professor, CSE

BUBT, Dhaka

Course Teacher

Capstone Defense Board

Department of Computer Science and Engineering, BUBT, Dhaka.

Dedication

We dedicate this project to our parents and families for their continuous support, patience, encouragement, and sacrifices throughout our academic journey. We also dedicate this work to our teachers and classmates whose guidance and feedback helped us complete the project. Finally, we dedicate this work to everyone who enjoys casual puzzle games and inspires us to create simple, engaging, and meaningful software.

Acknowledgement

We express our sincere gratitude to Almighty Allah for giving us the strength and patience to complete this project. We are thankful to Bangladesh University of Business and Technology (BUBT) and the Department of Computer Science and Engineering (CSE) for providing the academic environment and resources required for software development. We are also grateful to our course teacher for guidance, feedback, and evaluation throughout the project. We thank our teammates for their cooperation in UI design, programming, testing, debugging, asset preparation, and documentation. Finally, we appreciate the open-source Flutter, Firebase, and supporting software communities whose documentation and packages helped us build and refine the application.

Abstract

Think and Slide is a mobile sliding-puzzle game developed with Flutter. The project modernizes the classic sliding puzzle by combining a simple logical puzzle mechanic with a cozy, cartoon-inspired interface, multiple difficulty modes, progress tracking, daily streaks, global ranking, audio controls, vibration feedback, theme selection, and optional Google authentication. The puzzle engine represents the board as a list of tile positions and permits a tile to move only when it is orthogonally adjacent to the empty position. Boards are generated by performing legal random moves from a solved state, which preserves solvability. Game performance is measured using elapsed time and number of moves, and the resulting score is stored locally and, for authenticated users, synchronized with Cloud Firestore. Firebase Authentication is used for account access, while SharedPreferences provides local persistence for settings and progress. The completed application demonstrates how Flutter can be used to integrate game logic, responsive UI, multimedia feedback, local storage, authentication, and cloud data services in a single mobile application

Keywords: *Flutter; Sliding Puzzle; Mobile Game; Firebase; Firestore; Google Sign-In; SharedPreferences; Game UI; Daily Streak; Global Ranking.*

List of Figures

3.1 Figure 1: High-level architecture of Think and Slide. 6

3.2 Figure 2: Logical Firestore data relationship. 6

4.3 Figure 3: Think and Slide application logo used by the project. . . . 11

4.4 Figure 4: Think and Slide loading screen asset. 12

4.5 Figure 5: Primary home-screen visual theme. 12

4.6 Figure 6: Alternative home-screen visual theme.. . . . 13

List of Tables

1	Puzzle content organization in the project.	7
2	Main persistence responsibilities.	14
3	Qualitative comparison of feature coverage.	15
4	Major development milestones.	18

List of Abbreviations and Acronyms

API Application Programming Interface

CSE Computer Science and Engineering

GUI Graphical User Interface

IDE Integrated Development Environment

UI User Interface

UX Graphical User Interface

SDK Software Development Kit

CRUD Create, Read, Update, Delete

MP3 MPEG Audio Layer III

Firebase Google Firebase Platform

Firestore Cloud Firestore Database

Contents

Declaration	ii
Approval	iii
Dedication	iv
Acknowledgement	v
Abstract	vi
List of Abbreviations and Acronyms	ix
1 Introduction	1
1.1 Problem Statement	1
1.2 Problem Background	1
1.3 Objectives	2
1.4 Motivations	2
1.5 Significance of the Project	2
1.6 Project Contribution	3
1.7 Project Organization	3
2 Background Study	4
2.1 Existing Application Study	4
2.2 Existing Application Analysis	4
3 System Analysis and Design	6
3.1 ER/UML Diagram	6
3.2 Software Project Requirement	7
3.2.1 Data Collection and Preprocessing	7
3.2.2 Feature Selection	7
3.3 Analysis	8
3.4 Explanation of Different Modules	8

4	Implementation and Result Analysis	10
4.1	Environment Setup	10
4.2	App Implementation	10
4.3	Database Details	14
4.4	Result Analysis	14
4.5	Comparison with Other Existing Applications	15
5	Standards, Constraints, and Milestones	16
5.1	Sustainability Standards	16
5.2	Impacts on Society	16
5.3	Ethics	17
5.4	Challenges	17
5.5	Constraints	17
5.6	Timeline and Gantt Chart	18
6	Conclusion	19
6.1	Limitations	19
6.2	Future Works and Direction	20

Chapter 1

1 Introduction

Mobile games are widely used as short-form entertainment and as a way to exercise attention, planning, and problem-solving skills. The sliding puzzle is a particularly suitable game concept because its rules are easy to understand while its solutions require logical planning. Think and Slide was developed to provide a modern implementation of this classic puzzle with a friendly mobile-first interface. The application uses Flutter for cross-platform UI development and combines local storage, Firebase services, audio, vibration, and theme management to create a complete game experience.

1.1 Problem Statement

Traditional sliding puzzles are often presented with a plain board and limited feedback. A modern mobile player expects intuitive navigation, attractive visuals, responsive interaction, progress tracking, and optional personalization. The problem addressed by this project is therefore to design and implement a complete sliding puzzle application that preserves the core logical challenge while providing an engaging, accessible, and persistent mobile experience.

1.2 Problem Background

A standard sliding puzzle consists of numbered or image-based tiles and one empty position. A legal move occurs when a tile adjacent to the empty position is shifted into that position. Randomly arranging tiles without considering solvability can create impossible boards. Think and Slide avoids this issue by starting from a solved arrangement and repeatedly making legal moves to generate a shuffled but solvable state. The project also stores progress and settings so that the game can provide continuity between sessions.

1.3 Objectives

The major objectives are:

- Develop a mobile sliding-puzzle game using Flutter and Dart.
- Provide Easy, Medium, Hard, and Free To Play modes with image-based puzzles.
- Implement legal tile movement, shuffle, timer, move counter, win detection, and scoring.
- Provide a cozy cartoon-inspired interface with reusable animated menu buttons.
- Add local persistence for completed levels, time, moves, settings, themes, and streak data.
- Integrate Firebase Authentication and Cloud Firestore for user profiles, level progress, scores, and rankings.
- Provide Google Sign-In and a guest/local play path.
- Add background music, volume/mute controls, and vibration feedback.
- Provide daily streak and global ranking features to improve replay value.

1.4 Motivations

The main motivation was to transform a familiar puzzle into a more attractive and interactive mobile product. The project also provides practical experience in Flutter state management, asynchronous programming, local persistence, Firebase integration, multimedia handling, responsive UI construction, and collaborative software development.

1.5 Significance of the Project

The project demonstrates the integration of several software engineering concepts in one usable application. From an educational perspective, it connects algorithms and data structures with user-interface design and cloud services. From a user perspective, it provides short puzzle sessions with measurable performance through time, moves, score, and streaks.

1.6 Project Contribution

- A complete Flutter implementation of an image-based sliding puzzle.
- A solvability-preserving shuffle based on legal moves from the solved state.
- A reusable animated menu-button component with press feedback and sound.
- Persistent local progress and settings with cloud synchronization for signed-in users.
- Daily streak, level progression, score calculation, and global ranking features.
- Multiple visual themes, music control, and vibration settings for a personalized experience.

1.7 Project Organization

Chapter 1 introduces the project and its objectives. Chapter 2 studies existing puzzle and mobile-game approaches and identifies the design gap. Chapter 3 presents system analysis, requirements, architecture, data design, and the puzzle algorithm. Chapter 4 explains implementation, environment setup, database usage, interface modules, and result analysis. Chapter 5 discusses standards, social and ethical considerations, constraints, challenges, and milestones. Chapter 6 concludes the project and presents limitations and future directions.

Summary

Think and Slide addresses the need for a simple but polished sliding-puzzle experience by combining a solvable puzzle algorithm with modern mobile UI, persistent progress, social-style scoring, audio, vibration, and personalization. The following chapters describe how these requirements were analyzed and implemented.

Chapter 2

2 Background Study

The sliding puzzle is a classical state-space problem in which a board is transformed from an initial arrangement into a target arrangement through legal moves. Modern implementations commonly replace numbered tiles with images and add user interface features such as levels, hints, timers, scores, and progress tracking. Think and Slide follows this concept while focusing on a cozy visual identity and mobile interaction.

2.1 Existing Application Study

Existing casual puzzle applications demonstrate that simple rules can support long term engagement when combined with progression, feedback, and visual polish. Commercial puzzle games frequently use short sessions, level-based progression, sound effects, and reward systems. Board-game and puzzle applications also commonly use tile movement animations and tactile feedback because the feedback confirms that a move has been accepted. From a technical perspective, Flutter provides a reactive widget model and supports animation, asset management, audio packages, and platform APIs. Firebase provides authentication and cloud data services that are suitable for maintaining user profiles and scores. SharedPreferences is appropriate for small local values such as settings and level counters. These technologies form the technical background of the project.

2.2 Existing Application Analysis

A review of typical sliding-puzzle applications identifies several common limitations: basic visual presentation, limited customization, lack of persistent statistics, or separation between local progress and online rankings. Think and Slide attempts to combine these elements in a single application. The project supports multiple diffi-

culty modes, image collections, level locking, local statistics, cloud scores, a global ranking, a daily streak, music controls, vibration controls, and two selectable visual themes. The project also differs in its interaction design. The reusable MenuButton uses a short scale animation when pressed, a chunky shadow, rounded corners, and click audio. In the puzzle screen, valid tile movement produces a dedicated vibration response while invalid tile taps receive click feedback. These small interactions are intended to make the board feel responsive without adding unnecessary complexity

Summary

The background study shows that the core sliding-puzzle mechanic is simple, but a complete mobile product requires additional systems for progression, persistence, feedback, and personalization. Think and Slide addresses these needs through a combination of Flutter, local storage, Firebase, multimedia feedback, and a custom visual design

Chapter 3

3 System Analysis and Design

The system was designed as a Flutter mobile application with a layered structure. Screens handle presentation and navigation, reusable widgets handle common UI elements, services handle authentication, database operations, audio, streak logic, and theme persistence, and local assets provide backgrounds and puzzle images. Firebase is initialized at application startup and the authentication wrapper decides whether the user enters guest/local play, login, username setup, or the home screen.

image sample reference in figure 3.1

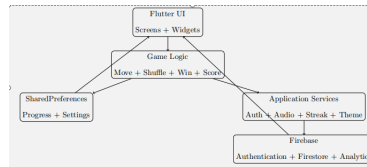


Figure 3.1: Figure 1: High-level architecture of Think and Slide.

3.1 ER/UML Diagram

The logical data model centers on the user. A Firestore user document stores identity-related fields, total score, and a progress map. Each progress entry is identified by difficulty and level and stores score, moves, and time. This structure allows the ranking screen to order users by total score while the game screen can update an individual level.

image sample reference in figure 3.2



Figure 3.2: Figure 2: Logical Firestore data relationship.

3.2 Software Project Requirement

Functional requirements include authentication and guest access, level selection, difficulty selection, image-based puzzle play, legal tile movement, timer, move counter, win detection, scoring, level progression, pause/restart, daily streak, global ranking, music control, vibration control, and theme selection. Non-functional requirements include responsive interaction, simple navigation, persistent settings, graceful handling of unavailable cloud data, low-latency tile interaction, maintainable modular code, and compatibility with Android devices supported by the Flutter project.

Table 1: Puzzle content organization in the project.

Mode	Configured Levels	Asset Source
Easy	10	assets/Easy/
Medium	10	assets/Medium/
Hard	10	assets/Hard/
Free To Play	21	assets/free to play/

3.2.1 Data Collection and Preprocessing

No machine-learning dataset is used. Instead, the project uses curated image assets organized into difficulty folders. The application resolves the appropriate image file for the selected level and divides the image conceptually into board positions through the tile grid. Image assets are packaged with the Flutter application so that gameplay does not depend on network access.

3.2.2 Feature Selection

The primary game-state variables are the tile list, row count, column count, empty tile position, move count, elapsed seconds, selected level, and difficulty. Supporting state includes theme, music volume, mute status, vibration status, login state, completed levels, and streak information

3.3 Analysis

The puzzle algorithm starts with a solved list containing tile indices from zero to the final empty position. It identifies the empty position and repeatedly selects one of its orthogonal neighbors at random. The selected neighbor is swapped with the empty position. Because every shuffle operation is itself a legal puzzle move, the resulting board remains reachable from the solved configuration. For a tile selected by the user, the application calculates the tile row and column and compares them with the empty cell row and column. A move is legal only when the two positions share a row or column and differ by exactly one cell. The selected tile is then exchanged with the empty position and the move counter is incremented. The completion test checks every board position against its solved index. When all positions match, the timer stops and the score is calculated using the implemented formula: $S = \max(0, \text{round}(100 - 0.25M - 0.05T))$, where M is the number of moves and T is elapsed time in seconds.

3.4 Explanation of Different Modules

- Authentication module: manages Google Sign-In, Firebase authentication state, guest/local access, and username setup.
- Home and navigation module: provides entry points to difficulty selection, daily streak, rankings, settings, and gameplay.
- Puzzle module: manages board state, legal moves, shuffle, timer, scoring, win detection, and progression.
- Progress module: stores local level completion and statistics and synchronizes authenticated progress with Firestore.
- Ranking module: reads users ordered by total score and displays a global ranking.
- Streak module: records puzzle completion dates and increments or resets the streak according to consecutive-day play.
- Audio and feedback module: manages background music, volume, mute state, click feedback, and vibration settings.

- Theme module: persists the selected visual theme and exposes it through a notifier so the UI can update immediately.

Summary

The design separates presentation, game logic, persistence, cloud services, and multimedia feedback. The central puzzle algorithm guarantees that generated boards are solvable, while the supporting modules provide the features expected from a complete mobile game.

Chapter 4

4 Implementation and Result Analysis

The implementation was completed as a Flutter project using Dart. The application initializes Firebase, loads theme and audio settings, enables immersive portrait gameplay, and starts from an authentication wrapper. The source tree is divided into screens, widgets, services, and theme management. Assets are packaged under dedicated image, difficulty, background, and music directories.

4.1 Environment Setup

The main development stack consists of Flutter and Dart, using the Flutter framework and its widget-based UI model, together with Android tooling and Firebase services for authentication, database, and analytics integration. The project uses documented Flutter and Firebase APIs as its application foundation Google, 2026a, 2026b. The project declares packages including firebase core, firebase auth, cloud firestore, firebase analytics, google sign in, shared preferences, audioplayers, and google fonts. Android configuration includes Firebase service configuration and Flutter’s Android build system. The application requests portrait orientation and immersive system UI mode. Background images are pre-cached on important screens to reduce visible loading delay during navigation. The puzzle image for the current level is also pre-cached before gameplay

4.2 App Implementation

The completed application contains the following major screens and interaction stages:

1. Splash/loading screen: presents the game identity while the application initializes.
2. Login/guest flow: allows Firebase-backed Google authentication or local/guest

access.

3. Username setup: collects a player identity when required by the profile flow.
4. Home screen: provides the main navigation for gameplay and supporting features.
5. Difficulty selection: offers Easy, Medium, Hard, and Free To Play modes.
6. Level selection: displays level availability, completed progress, and recorded moves/time.
7. Gameplay: renders the sliding board, timer, moves, controls, and image puzzle.
8. Pause menu: provides resume, restart, settings, and home actions.
9. Congratulations screen: displays completion time, moves, and score and offers the next puzzle.
10. Daily streak: shows consecutive puzzle-solving activity.
11. Global ranking: displays authenticated players ordered by total score.
12. Settings: controls music volume, mute, vibration, account access, and theme selection.



Figure 4.3: Figure 3: Think and Slide application logo used by the project.



Figure 4.4: Figure 4: Think and Slide loading screen asset.



Figure 4.5: Figure 5: Primary home-screen visual theme.



Figure 4.6: Figure 6: Alternative home-screen visual theme..

4.3 Database Details

Cloud Firestore stores user information in a users collection. A user document contains identity information and a totalScore. Level-specific records are maintained in a progress map using keys such as difficulty and level. Each entry stores score, moves, and time. The ranking service orders user documents by totalScore in descending order. SharedPreferences is used for local persistence, following the package’s intended role for simple key-value application settings Flutter Team, 2026. It stores completed level counters, per-level moves and time, music volume, mute status, vibration status, theme selection, guest state, and locally cached progress. When a guest later authenticates, the local progress can be synchronized to the cloud account.

Table 2: Main persistence responsibilities.

Storage	Purpose
SharedPreferences	Local level completion, moves, time, score, streak, theme, audio and vibration settings.
Firebase Authentication	Google-based identity and authentication state.
Cloud Firestore	User profile, total score, per-level progress, and global ranking data.
Firebase Analytics	Application open event and analytics integration.

4.4 Result Analysis

Functional testing focused on the main user journey: application launch, authentication/guest entry, difficulty selection, level selection, puzzle movement, completion, score calculation, persistence, ranking, settings, and daily streak. The implemented game also handles invalid tile taps by providing click feedback without changing the board. Valid moves update the board and move counter and trigger the configured slide vibration. The shuffle process was designed to preserve solvability. The scoring system rewards fewer moves and less time, while never returning a negative score. Level completion is stored locally and synchronized to Firestore when an authenticated user is available.

4.5 Comparison with Other Existing Applications

Compared with a minimal classic sliding-puzzle implementation, Think Slide provides a broader feature set. The comparison is qualitative because no external application was benchmarked under identical hardware and test conditions.

Table 3: Qualitative comparison of feature coverage.

Feature	Classic Puzzle	Typical Simple Mobile Puzzle	Think and Slide
Image puzzles	Limited	Often	Yes
Multiple difficulty modes	Sometimes	Often	Yes
Level progression	Limited	Yes	Yes
Local progress	Rare	Sometimes	Yes
Cloud progress	Rare	Sometimes	Yes
Global ranking	Rare	Sometimes	Yes
Daily streak	Rare	Sometimes	Yes
Theme selection	Rare	Sometimes	Yes
Music and vibration settings	Limited	Often	Yes
Custom animated UI	Limited	Varies	Yes

Summary

The implementation integrates the puzzle algorithm with a complete mobile application architecture. Testing and code inspection show that the main game flow, persistence, scoring, cloud synchronization, multimedia controls, and progression systems are represented in the completed project. The result is a feature-rich casual puzzle application rather than only a standalone puzzle board.

Chapter 5

5 Standards, Constraints, and Milestones

The development of Think Slide followed practical software engineering principles: modular code organization, reusable widgets, separation of service responsibilities, persistent configuration, and validation of user actions before modifying game state. The project also considers usability, maintainability, ethical asset use, and mobile resource constraints.

5.1 Sustainability Standards

The project uses a lightweight local-first gameplay model. Puzzle images are bundled with the application, so the core game does not require continuous network access. Cloud communication is reserved for authentication, progress synchronization, ranking, and related services. Reusable Flutter widgets and centralized services reduce duplicate code and make future maintenance easier. Audio and image assets are kept in organized directories so that unnecessary resources can be removed or replaced without changing the puzzle engine.

5.2 Impacts on Society

The application is intended as casual entertainment and a small cognitive challenge. It encourages planning and attention through a simple logical task. The daily streak and ranking features can increase motivation, but they should be used responsibly and without manipulative reward mechanisms. The project can also serve as an educational example of how game logic and cloud services can be integrated in a mobile application.

5.3 Ethics

The project should use original, licensed, or permission-granted visual and audio assets. The visual direction is described as Ghibli-inspired rather than claiming official Studio Ghibli ownership. User account data should be handled only for features that require it, and cloud data should not be exposed unnecessarily. Ranking should avoid displaying sensitive personal information. Future production deployment should also include a clear privacy policy and appropriate consent where required.

5.4 Challenges

Major implementation challenges included maintaining solvable puzzle states, coordinating asynchronous Firebase operations with the Flutter UI, keeping local and cloud progress consistent, handling guest-to-authenticated synchronization, preventing audio from continuing incorrectly when the app is paused, and maintaining responsive animations. Asset path consistency was also important because Flutter requires declared asset directories in `pubspec.yaml`.

5.5 Constraints

Design constraints: The application is portrait-oriented and follows a compact mobile layout. The interface uses a cartoon/cozy visual language and must remain readable over image backgrounds. Component constraints: The project depends on Flutter packages and Firebase services whose APIs and platform support can change over time. The application also relies on device audio and vibration capabilities. Performance constraints: Large image assets can increase application size and memory usage, so image pre-caching and organized asset management are important. Connectivity constraints: Core puzzle play is local, but cloud ranking and synchronization depend on network availability. Budget constraints: The project was developed primarily with student-accessible tools, open-source packages, local assets, and Firebase services. Production-scale hosting, marketing, analytics, and commercial asset licensing may introduce additional costs.

5.6 Timeline and Gantt Chart

The project development can be represented in the following milestone sequence:

Table 4: Major development milestones.

Task	M1	M2	M3	M4	M5
Requirement and concept	X				
UI/Figma and assets	X	X			
Puzzle engine		X	X		
Firebase/auth/progress			X	X	
Testing and debugging				X	X
Documentation/report					X

Summary

The project was developed under practical academic constraints while maintaining a focus on modularity, usability, asset licensing, and resource efficiency. The milestone plan reflects an iterative process in which interface design, game logic, cloud services, testing, and documentation were developed and refined progressively.

Chapter 6

6 Conclusion

Think and Slide successfully implements a modern mobile sliding-puzzle experience using Flutter. The completed application combines a solvability-preserving shuffle algorithm, legal tile movement, timer, move counter, score calculation, level progression, image-based puzzles, animated interface elements, audio, vibration, theme selection, daily streaks, and ranking. Firebase Authentication and Cloud Firestore extend the local game with account-based progress and global scores, while Shared-Preferences keeps small settings and progress available locally. The project demonstrates that a relatively simple game mechanic can become a complete software product when supported by appropriate architecture, persistence, interaction feedback, and user-centered design. It also provided practical experience in Dart/Flutter development, asynchronous operations, cloud integration, multimedia handling, debugging, asset management, and mobile UI design.

6.1 Limitations

The current version has several limitations. The puzzle content is based on a finite collection of bundled images, so the content library must be expanded manually. The scoring formula is fixed and does not yet adapt to puzzle size or difficulty. The ranking system depends on Firestore connectivity and the quality of the stored user data. The project also does not yet include advanced anti-cheating validation for online scores. In addition, the current application is primarily optimized for portrait mobile use rather than tablets or landscape layouts. The visual style and assets may also require additional licensing review before commercial publication. Finally, while the application includes basic analytics initialization, it does not yet provide a comprehensive product analytics dashboard for retention, level difficulty, or user behavior analysis.

6.2 Future Works and Direction

Future development can extend the game with larger puzzle sizes, custom image importing, additional themes, hint systems, undo support, sound-effect packs, smoother tile transitions, accessibility options, and more sophisticated achievement systems. The scoring model could be normalized by board size and difficulty. Online leaderboards could be strengthened with server-side validation and seasonal ranking. A future release could also add cloud-backed puzzle content so that new puzzle packs can be delivered without requiring a full application update. Monetization options such as optional advertisements or cosmetic purchases could be considered carefully if the project is published commercially. Finally, a broader testing phase across different Android devices would improve performance, accessibility, and compatibility before Play Store release.

Summary

Think and Slide meets its core objective of delivering an engaging, feature-rich sliding puzzle game in Flutter. The current architecture provides a strong foundation for future content, gameplay, social, and commercial improvements while preserving the simplicity of the original puzzle concept.